

MP Lab: Perfect Assembly Programs

Handwritten Code Reconstruction

1 Fibonacci Series Generation

Program: Generates a Fibonacci series of length n using 8086 Assembly.

```
; Q. Write an ALP to generate fibonacci series of length n
```

```
DATA SEGMENT
```

```
F1 DB 00
```

```
F2 DB 01
```

```
F3 DB ?
```

```
N DB 10
```

```
MI DB 0DH, 0AH, 'the fibonacci seriesis : $'
```

```
DATA ENDS
```

```
CODE SEGMENT
```

```
ASSUME CS:CODE, DS:DATA
```

```
START : MOV AX, DATA
```

```
        MOV DS, AX
```

```
        MOV AH, 09H
```

```
        LEA DX, MI
```

```
        INT 21H
```

```
        MOV BL, F1
```

```
        CALL DISPHEX
```

```
        MOV DL, ' '
```

```
        MOV AH, 02H
```

```
        INT 21H
```

```
        MOV BL, F2
```

```
        CALL DISPHEX
```

```
        MOV DL, ' '
```

```
        MOV AH, 02H
```

```
        INT 21H
```

```
        MOV CL, 00
```

```
        SUB N, 02
```

```
UP :    CMP CL, N
```

```
        JZ EXIT
```

```
        MOV AL, F1
```

```
        ADD AL, F2
```

```
        MOV BL, AL
```

```
        CALL DISPHEX
```

```
        MOV DL, ' '
```

```
        MOV AH, 02H
```

```

        INT 21H
        MOV F3, BL
        MOV AL, F2
        MOV F1, AL
        MOV AL, F3
        MOV F2, AL
        INC CL
        JMP UP
EXIT :   MOV AH, 4CH
        INT 21H

DISPHEX PROC NEAR
        MOV DL, BL
        MOV CL, 04
        SHR DL, CL
        CMP DL, 09H
        JBE DOWN
        ADD DL, 07H
DOWN :   ADD DL, 30H
        MOV AH, 02H
        INT 21H
        MOV DL, BL
        AND DL, 0FH
        CMP DL, 09H
        JBE DOWN1
        ADD DL, 07H
DOWN1 :  ADD DL, 30H
        MOV AH, 02H
        INT 21H
        RET
DISPHEX ENDP
CODE ENDS
END START

```

Expected Output

the fibonacci series is : 00 01 01 02 03 05 08 0D 15 22

2 Palindrome String Check

Program: Accepts a string, reverses it, and checks if it is a palindrome.

```
; Q. Write an ALP to accept a string as sequence of characters,  
; reverse and check palindrome or not
```

Data Segment

```
Str1 DB 50 DUP(?)  
Str2 DB 50 DUP(?)  
n DB ?  
M1 DB 0DH,0AH, ' Enter a String: $'  
M2 DB 0DH,0AH, ' It is palindrome $'  
M3 DB 0DH,0AH, ' It is not a palindrome$'  
Data ends
```

Code Segment

Assume cs:code, ds:Data

```
Start: MOV AX, DATA  
      MOV DS, AX  
      CALL MSG1  
      LEA SI, Str1  
      LEA DI, Str2  
      CALL Readstring  
      MOV n, CL  
      DEC SI  
      CALL Reverse  
      LEA SI, Str1  
      LEA DI, Str2  
      CALL Check  
      MOV AH, 4CH  
      INT 21H
```

```
MSG1 Proc Near  
      MOV AH, 09H  
      LEA DX, M1  
      INT 21H  
      RET
```

MSG1 ENDP

Readstring Proc Near

```
      MOV CL, 00  
UP:    CMP CL, 50  
      JZ Stop  
      MOV AH, 01H  
      INT 21H  
      CMP AL, 0DH  
      JZ Stop  
      MOV [SI], AL  
      INC SI  
      INC CL
```

```

        JMP UP
Stop: Ret
Readstring ENDP

Reverse Proc Near
        MOV CL, 00
UP1:    CMP CL, n
        JZ Stop1
        MOV BL, [SI]
        MOV [DI], BL
        INC DI
        DEC SI
        INC CL
        JMP UP1
Stop1: Ret
Reverse Endp

Check proc Near
        MOV CL, 00
Up2:    CMP CL, n
        JZ Pal
        MOV BL, [DI]
        CMP [SI], BL
        JNE NotPal
        INC SI
        INC DI
        INC CL
        JMP Up2
Pal:    MOV AH, 09H
        LEA DX, M2
        INT 21H
        JMP Exit
NotPal: MOV AH, 09H
        LEA DX, M3
        INT 21H
Exit: Ret
Check Endp

Code ends
End Start

```

Expected Output

Case 1:

```
Enter a String: hello  
It is not a palindrome
```

Case 2:

```
Enter a String: MADAM  
It is palindrome
```

3 Binary Search Algorithm

Program: Searches for an 8-bit key in a sorted array using Binary Search.

```
; Q. Write an ALP to search a key element in an array of n 8-bit  
; numbers using binary search algorithm.
```

DATA SEGMENT

```
ARR DB 10H, 20H, 30H, 40H, 50H  
KEY DB 40H  
LEN DB $ - ARR  
MSG1 DB 0DH, 0AH, ' Key found $'  
MSG2 DB 0DH, 0AH, ' Key not found $'  
MID DB ?  
DATA ENDS
```

CODE SEGMENT

```
ASSUME CS:CODE, DS:DATA
```

```
START: MOV AX, DATA  
        MOV DS, AX  
        LEA SI, ARR  
        MOV AL, LEN  
        DEC AL  
        MOV AH, 00  
        ADD AX, SI  
        MOV DI, AX  
        MOV CL, KEY  
UP:     CMP SI, DI  
        JG Notfound  
        MOV AX, SI  
        ADD AX, DI  
        SHR AX, 01  
        MOV BX, AX  
        MOV MID, BX  
        CMP CL, [BX]  
        JZ Found  
        JB lefthalf  
        INC WORD PTR MID  
        MOV SI, MID  
        JMP UP  
lefthalf: DEC WORD PTR MID  
          MOV DI, MID  
          JMP UP  
Notfound: Lea DX, MSG2  
          MOV AH, 09H  
          INT 21H  
          JMP Exit  
found:   Lea DX, MSG1  
          MOV AH, 09H  
          INT 21H  
Exit:    MOV AH, 4CH
```

```
INT 21H
CODE ENDS
END START
```

Expected Output

Key found